League of Legends

Developer API Policy

Before you begin, read through the General and Game Policies, Terms of Use and Legal Notices. Developers must adhere to policy changes as they arise.

When developing using the API, you must abide by the following:

- Products cannot violate any laws.
- Do not create or develop games utilizing Riot's Intellectual Property (IP).
- No cryptocurrencies or no blockchain.
- No apps serving as a "data broker" between our API and another third-party company.
- Products cannot closely resemble Riot's games or products in style or function.
- Only the following Riot IP assets may be used in the development and marketing of your product:
    - Press kit
    - Example: Using Riot logos and trademarks from the Press Kit must be limited to cases where such use is unavoidable in order to serve the core value of the product.
    - Game-Specific static data
- You must post the following legal boilerplate to your product in a location that is readily visible to players:
    - [Your Product Name] is not endorsed by Riot Games and does not reflect the views or opinions of Riot Games or anyone officially involved in producing or managing Riot Games properties. Riot Games and all associated properties are trademarks or registered trademarks of Riot Games, Inc

Registration

If your product serves players, you must register it with us regardless of whether or not your product uses official documented APIs. You must make sure its description and metadata are kept up to date with the current version of your product.

Monetization

To monetize your product, you must abide by the following:

- Your product cannot feature betting or gambling functionality.
- Your product must be registered on the Developer Portal and your product status is either Approved or Acknowledged.
- You must have a free tier of access for players, which may include advertising.
- Your content must be transformative if you are charging players for it.
    - What is transformative?

        - Was value added to the original by creating new information, new aesthetics, new insights, and understandings? If so, then it was transformative.
    - Acceptable ways to charge players are:
        - Subscriptions, donations, or crowdfunding.
        - Entry fees for tournaments.
        - Currencies that cannot be exchanged back into fiat.
    - Your monetization cannot gouge players or be unfair, as decided by Riot.

If you are unsure if your monetization platform is acceptable, contact us through the Developer Portal.

Security

You must adhere to the following security policies:

- Do not share your Riot Games account information with anyone.
- Do not use a Production API key to run multiple projects. You may only have one product per key.
- Use SSL/HTTPS when accessing the APIs so your API key is kept safe.
- Your API key may not be included in your code, especially if you plan on distributing a binary.
    - This key should only be shared with your teammates. If you need to share an API key for your product with teammates, make sure your product is owned by a group in the Developer portal. Add additional accounts to that account as needed.

Game Integrity

- Products cannot alter the goal of the game (i.e. Destroy the Nexus).
- Products cannot create an unfair advantage for players, like a cheating program or giving some players an advantage that others would not otherwise have.
- Products should increase, and not decrease the diversity of game decisions (builds, compositions, characters, decks).
- Products should not remove game decisions, but may highlight decisions that are important and give multiple choices to help players make good decisions.
- Products cannot create alternatives for official skill ranking systems such as the ranked ladder. Prohibited alternatives include MMR or ELO calculators.
- Products cannot identify or analyze players who are deliberately hidden by the game.

Tournament Policies

- Tournaments must:
    - Follow all monetization policies above.

    - Allot at least 70% of the entry fees to the prize pool.
    - Win conditions must be fair and transparent to players (we determine fair).
    - Must have at least 20 participants.
    - Not include any gambling.
- If you are a tournament organizer operating in the US or Canada please refer to, and adhere to, these North American tournament organizer policies.
- If you are a tournament organizer operating in Oceania please refer to, and adhere to, these Oceanic tournament organizer policies.
- If you are a tournament organizer operating in Europe, please refer to, and adhere to, these European tournament organizer policies.

Summoner Names to Riot IDs

On November 20, 2023, we are transitioning our systems away from Summoner Names to using Riot ID as an authoritative way to reference players in League and TFT starting later this year. As such, you will need to make an update to the applicable API. Details for this transition can be found below.

All player-facing front-end fields and forms will require modification. Applications featuring the "Find your Summoner by Region + Name" functionality must adapt to locate summoners using Riot IDs, which are formed by combining the "game name" and "tag line".

For all other Riot API endpoints, filtering by player can be accomplished using either the PUUID or summonerID. Some APIs offer both options, but we recommend employing PUUID endpoints when available.

We recommend no longer using these endpoints (deprecated):

- https://developer.riotgames.com/apis#summoner-v4/GET_getBySummonerName - /lol/summoner/v4/summoners/by-name/{summonerName}
- https://developer.riotgames.com/apis#tft-summoner-v1/GET_getBySummonerName - /tft/summoner/v1/summoners/by-name/{summonerName}

Although deprecated, they can still be used to convert Summoner Names to PUUID or summonerID. However, we discourage using them as part of your application since they will be removed in the future.

Obtaining PUUID and summonerID from RiotID

- (ACCOUNT-V1) https://developer.riotgames.com/apis#account-v1/GET_getByRiotId - Utilize the endpoint /riot/account/v1/accounts/by-riot-id/{gameName}/{tagLine} to obtain the PUUID associated with a given account by Riot ID (gameName + tagLine).
- (SUMMONER-V4) https://developer.riotgames.com/apis#summoner-v4/GET_getByPUUID - Access the endpoint /lol/summoner/v4/summoners/by-puuid/{encryptedPUUID} to retrieve summoner data by PUUID, including summonerID.

Obtaining Riot ID from PUUID

For third-party apps, displaying Riot IDs in place of summoner names within frontend fields is now necessary. If you lack a Riot ID for a particular player in your database or wish to keep it up to date, you can acquire it through the following endpoints:

- (ACCOUNT-V1) https://developer.riotgames.com/apis#account-v1/GET_getByPuuid - Use the endpoint /riot/account/v1/accounts/by-puuid/{puuid} to fetch account information (gameName + tagLine) by PUUID.

Obtaining Riot ID from summonerID

In cases where you do not possess a PUUID for a player, you can employ the player's summonerID to obtain the PUUID:

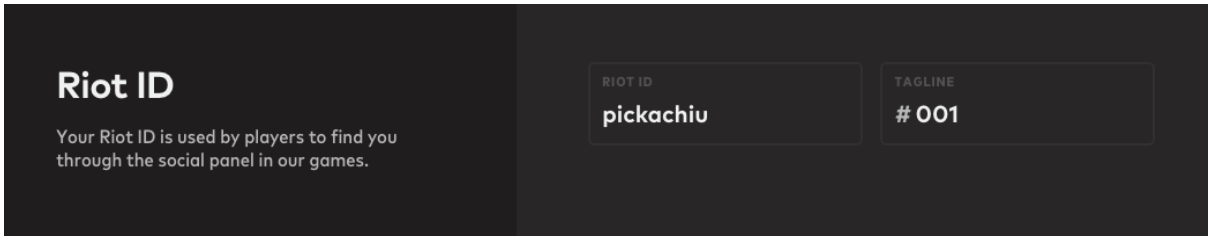
- (SUMMONER-V4) https://developer.riotgames.com/apis#summoner-v4/GET_getBySummonerId - Access the endpoint /lol/summoner/v4/summoners/{encryptedSummonerId} to retrieve summoner data by summonerID, which can be used to obtain the corresponding PUUID.

Summoner Names Post Migration

Following this migration, Summoner Names endpoints will remain accessible. However, they will no longer be player-facing. We intend to keep them temporarily to avoid disrupting existing APIs. For summoners created after this transition, they will be assigned a random uuidv4 generated string.

We strongly advise utilizing this deprecation period to refactor your existing applications and remove reliance on the summoner names field. In future releases, we will remove summoner names from the API altogether.

Example of a RiotID input:



Check our FAQ for more details!

Game Policy

Use Cases for Production Keys

Production keys are meant for larger-size, professional projects.

Riot analyzes two main factors when evaluating applications:

- Is the use case good and approved?
- Does the developer show they will deliver on that use case?

To demonstrate that your app meets the use case, you should be able to have one or more of the following:

- Be an established brand that wants to add Riot Games to its portfolio.
- New app that is fully functional and testable by Riot.

- Prototype that is mostly testable by Riot.
- Mockups where Riot can clearly express your intent and the user flow.
- A deck that shows your ambition and intent and some of the user flow.

Riot needs to see the user flow to understand what your intended player experience is, such as account creation process, login pipeline, or queuing up for match pipeline.

You must also send a link to a working site, mockup, prototype, or rendering where it is easy to understand the user flows of the tool.

Examples of Approved Use Cases for Personal Keys

Personal keys are meant for smaller-size, personal projects.

- Personal sites.
- School projects.
- Creating a proof of concept for a Production Key request
- Examples of Approved Use Cases for Production Keys.
- Showing (self) player stats
- Running tournaments.
- Training tools that allow players to view their own match histories and aggregate stats.
- Looking For Game (LFG) tools.
- Game overlays that provide static data that is available prior to the game.
- Aggregate player stats (no specific players).
- Official Ladder Leaderboards.

Examples of Unapproved Use Cases

The following use cases will not be approved:

- Products cannot display win rates for Augments or Arena Mode items. This applies to all websites, applications and overlays.
- Products may not provide any game-session-specific information that would be previously unknown to the player.
- Apps that dictate player decisions.
- Apps that violate the general game policies.
- Products may not publicly display a player's match history from the custom match queue unless the player opts in to share this specifically for League of Legends. Otherwise, a player’s custom match data may only be made available to them using RSO.

RSO Integration

RSO or Riot Sign On, allows players to safely link their Riot Account to other applications. This access is only available to developers with Production Level API Keys.

Getting a Production Key

Before you can get started with RSO, you will need a production key. If you do not have one, please create one at https://developer.riotgames.com after reading our policies. If approved, we will contact you in the developer portal app messaging to kick off the RSO integration process.

Implementing RSO

RSO - Client Secret Basic - Private Key JWT

Sample RSO Node App: https://static.developer.riotgames.com/docs/rso/rso-example-app.zip

Players should be directed to login at this link: https://auth.riotgames.com/authorize?client_id={your-client-id}&redirect_uri={your-redirect-uri}&response_type=code&scope=openid+offline_access.

After logging in, players are redirected back to the redirect_uri you specified. See Implementing Riot Sign On and Example RSO Node App for information about how to integrate with RSO and to view a sample Node web server that implements the example.

Your RSO client has access to endpoints that will allow you to identify who logged in.

For Legends of Legends, you will use /riot/account/v1/accounts/me:

curl --location --request GET 'https://americas.api.riotgames.com/riot/account/v1/accounts/me' --header 'Authorization: Bearer {accessToken}'

curl --location --request GET 'https://europe.api.riotgames.com/riot/account/v1/accounts/me' --header 'Authorization: Bearer {accessToken}'

curl --location --request GET 'https://asia.api.riotgames.com/riot/account/v1/accounts/me' --header 'Authorization: Bearer {accessToken}'

The accounts data return from each cluster is identical. We recommend using the cluster closest to your servers.

Routing Values

To execute a request to the League of Legends (LoL) API, you must select the correct host to execute your request to. LoL API uses routing values in the domain to ensure your request is properly routed. Platform IDs and regions as routing values, such as na1 and americas. Routing values are determined by the topology of the underlying services. Services are frequently clustered by platform resulting in platform IDs being used as routing values. Services may also be clustered by region, which is when regional routing values are used. The best way to tell if an endpoint uses a platform or a region as a routing value is to execute a sample request through the Reference page.

TH2 and PH2 servers have been merged into the SG2 server. This server merge is part of our ongoing effort to enhance the gaming experience for Teamfight Tactics (TFT) and League of Legends (LoL) players in Southeast Asia (SEA).

Platform Routing Values

Platform	Host
BR1	br1.api.riotgames.com
EUN1	eun1.api.riotgames.com
EUW1	euw1.api.riotgames.com
JP1	jp1.api.riotgames.com
KR	kr.api.riotgames.com
LA1	la1.api.riotgames.com
LA2	la2.api.riotgames.com
NA1	na1.api.riotgames.com
OC1	oc1.api.riotgames.com
TR1	tr1.api.riotgames.com
RU	ru.api.riotgames.com
SG2	sg2.api.riotgames.com
TW2	tw2.api.riotgames.com
VN2	vn2.api.riotgames.com

Regional Routing Values

Region	Host
AMERICAS	americas.api.riotgames.com
ASIA	asia.api.riotgames.com

Region	Host
EUROPE	europe.api.riotgames.com
SEA	sea.api.riotgames.com

Data Dragon

Data Dragon is our way of centralizing League of Legends game data and assets, including champions, items, runes, summoner spells, and profile icons. All of which can be used by third-party developers. You can download a compressed tarball (.tgz) for each patch that contains all assets for that patch. Updating Data Dragon after each League of Legends patch is a manual process, so it is not always updated immediately after a patch.

Latest
https://ddragon.leagueoflegends.com/cdn/**dragontail-9.19.1.tgz**

Patch 10.10 was uploaded as a zip archive (.zip) instead of the typical compressed tarball (.tgz)
https://ddragon.leagueoflegends.com/cdn/**dragontail-10.10.5.zip**

Versions

You can find all valid Data Dragon versions in the versions file. Typically there is only a single build of Data Dragon for a given patch, however, there may be additional builds. This typically occurs when there is an error in the original build. As such, you should always use the most recent Data Dragon version for a given patch for the best results.

https://ddragon.leagueoflegends.com/api/**versions.json**

Regions

Data Dragon versions are not always equivalent to the League of Legends client version in a region. You can find the version each region is using in the realms files.

https://ddragon.leagueoflegends.com/realms/**na.json**

Data & Assets

Data Dragon provides two kinds of static data: data files and game assets. The data files provide raw static data on various components of the game such as summoner spells, champions, and items. The assets are images of the components described in the data files.

Data Files

The data file URLs include both a version and language code. The examples in the documentation below use version 9.19.1 and the en_US language code. If you want to view assets released in other versions or languages, replace the version or language code in the URL.

Languages

Data Dragon provides localized versions of each of the data files in languages supported by the client. Below is a list of the languages supported by Data Dragon, which you can also retrieve from the Data Dragon languages file.

https://ddragon.leagueoflegends.com/cdn/**languages.json**

CODE	LANGUAGE
cs_CZ	Czech (Czech Republic)
el_GR	Greek (Greece)
pl_PL	Polish (Poland)
ro_RO	Romanian (Romania)
hu_HU	Hungarian (Hungary)
en_GB	English (United Kingdom)
de_DE	German (Germany)
es_ES	Spanish (Spain)
it_IT	Italian (Italy)
fr_FR	French (France)
ja_JP	Japanese (Japan)
ko_KR	Korean (Korea)
es_MX	Spanish (Mexico)
es_AR	Spanish (Argentina)
pt_BR	Portuguese (Brazil)
en_US	English (United States)
en_AU	English (Australia)

CODE	LANGUAGE
ru_RU	Russian (Russia)
tr_TR	Turkish (Turkey)
ms_MY	Malay (Malaysia)
en_PH	English (Republic of the Philippines)
en_SG	English (Singapore)
th_TH	Thai (Thailand)
vi_VN	Vietnamese (Viet Nam)
id_ID	Indonesian (Indonesia)
zh_MY	Chinese (Malaysia)
zh_CN	Chinese (China)
zh_TW	Chinese (Taiwan)

Champions

There are two kinds of data files for champions. The champion.json data file returns a list of champions with a brief summary. The individual champion JSON files contain additional data for each champion.

https://ddragon.leagueoflegends.com/cdn/9.19.1/data/en_US/**champion.json**
https://ddragon.leagueoflegends.com/cdn/9.19.1/data/en_US/champion/**Aatrox.json**

Interpreting Spell Text

Lore, tips, stats, spells, and even recommended items are all part of the data available for every champion. Champion spell tooltips often have placeholders for variables that are signified by double curly brackets. Here are some tips about interpreting these placeholders:

{{ eN }} placeholders
Placeholders are replaced by the corresponding item in the array given in the effectBurn field. For example, {{ eN }} is a placeholder for spell["effectBurn"]["1"].

/* Amumu's Bandage Toss */

"tooltip": "Launches a bandage in a direction. If it hits an enemy unit, Amumu pulls himself to them, dealing {{ e1 }} <scaleAP>(+{{ a1 }})</scaleAP> magic damage and stunning for {{ e2 }} second.",

"effectBurn": [

null,

"80/130/180/230/280",

"1",

"1350",

...

]

{{ aN }} or {{ fN }} placeholders

These placeholders are slightly more complicated. Their values can be found in the vars field. First, find the object in the vars array whose key matches the variable. For example, for {{ a1 }}, find the object in the vars array whose key field has the value a1. The value for this variable is the coeff field in that same object.

/* Amumu's Bandage Toss */

"tooltip": "Launches a bandage in a direction. If it hits an enemy unit, Amumu pulls himself to them, dealing {{ e1 }} <scaleAP>(+{{ a1 }})</scaleAP> magic damage and stunning for {{ e2 }} second.",

"vars": [

{

"key": "a1",

"link": "spelldamage",

"coeff": [

0.7

]

}

]

Under a champions spells there are two fields effect and effectBurn. effect contains an array of an ability's values per level where, in contrast, effectBurn contains a string of all the values at every level. (e.g., "effect": [30,60,90,120,150] vs "effectBurn": "30/60/90/120/150"). You might notice how the effect and effectBurn arrays have a null value in the 0 index. This is because those values are taken from designer-facing files where arrays are 1-based. JSON is 0-based so a null is inserted to make it easier to verify the JSON files are correct.

"effect": [

null,

[120, 150, 180, 210, 240],

```
  [ 50, 70, 90, 110, 130 ],
  [ 25, 35, 45, 55, 65 ],
  [ 0.2, 0.2, 0.2, 0.2, 0.2 ],
  [ 50, 60, 70, 80, 90 ]
],
"effectBurn": [
  "",
  "120/150/180/210/240",
  "50/70/90/110/130",
  "25/35/45/55/65",
  "0.2",
  "50/60/70/80/90"
]
```

Calculating Spell Costs

In most cases a spell costs mana or energy, you will find those related costs under the cost and costBurn fields. When a spell costs health, the cost will be found in the effect and effectBurn fields. You can determine how to calculate the cost of a spell by looking at the resource field, which should point you to the variable being used to display the cost of a spell.

```
/* Soraka's Astral Infusion */
"resource": "10% Max Health, {{ cost }} Mana",
"cost": [ 20, 25, 30, 35, 40 ],
"costBurn": "20/25/30/35/40"
/* Shen's Vorpal Blade */
"resource": "{{ cost }} Energy",
"cost": [ 60, 60, 60, 60, 60 ],
"costBurn": "60"
/* Dr. Mundo's Infected Cleaver */
"resource": "{{ e3 }} Health",
"cost": [ 0, 0, 0, 0, 0 ],
"costBurn": "0",
"effect": [
  null,
```

[80, 130, 180, 230, 280],

[15, 18, 21, 23, 25],

[50, 60, 70, 80, 90],

[40, 40, 40, 40, 40],

[2, 2, 2, 2, 2]

],

"effectBurn": [

"",

"80/130/180/230/280",

"15/18/21/23/25",

"50/60/70/80/90",

"40",

"2"

]

Champion Splash Assets

https://ddragon.leagueoflegends.com/cdn/img/champion/splash/Aatrox_**0**.jpg

The number at the end of the filename corresponds to the skin number. You can find the skin number for each skin in the file for each individual champion in Data Dragon. Each champion contains a skins field and the skin number is indicated by the num field.

/* Aatrox (id: 266) */

"skins": [

{

"id": 266000,

"name": "default",

"num": 0

},

{

"id": 266001,

"name": "Justicar Aatrox",

"num": 1

},

{

"id": 266002,

"name": "Mecha Aatrox",

"num": 2

}

]

Champion Loading Screen Assets

https://ddragon.leagueoflegends.com/cdn/img/champion/loading/Aatrox_**0**.jpg
The number at the end of the filename follows the same convention described in the Champion Splash Art.

Champion Square Assets

https://ddragon.leagueoflegends.com/cdn/9.19.1/img/champion/**Aatrox**.png

Champion Passive Assets

https://ddragon.leagueoflegends.com/cdn/9.19.1/img/passive/**Anivia_P**.png

You can find the filename for each champion's passive in the individual champion Data Dragon file. The JSON contains a passive field with image data. The filename is indicated by the full field.

/* Anivia (id: 34) */

"passive": {

"name": "Rebirth",

"description": "Upon dying, Anivia will revert into an egg. If the egg can survive for six seconds, she is gloriously reborn.",

"image": {

"full": "Anivia_P.png",

"sprite": "passive0.png",

"group": "passive",

"x": 240,

"y": 0,

"w": 48,

"h": 48

}

}

}

Champion Ability Assets

https://ddragon.leagueoflegends.com/cdn/9.19.1/img/spell/**FlashFrost.png**

You can find the file name for each champion's abilities in the individual champion Data Dragon file. The spells field contains an array of objects which includes image data. The filename is indicated by the full field.

/* Anivia (id: 34) */

"spells": [

{

"id": "FlashFrost",

"name": "Flash Frost",

"description": "Anivia brings her wings together and summons a sphere of ice that flies towards her opponents, chilling and damaging anyone in its path. When the sphere explodes it does moderate damage in a radius, stunning anyone in the area.",

"image": {

"full": "FlashFrost.png",

"sprite": "spell0.png",

"group": "spell",

"x": 192,

"y": 144,

"w": 48,

"h": 48

}

},

...

]

Items

Data Dragon also provides the same level of detail for every item in the game. Within Data Dragon, you can find info such as the item's description, purchase value, sell value, items it builds from, items it builds into, and stats granted from the item.

https://ddragon.leagueoflegends.com/cdn/9.19.1/data/en_US/**item.json**

The effect field holds an array of variables used extra scripts. For example, on Doran's shield you see the following data in the effect field, which corresponds to the 8 damage that is blocked from champion attacks.

"effect": {

"Effect1Amount": "8"

}

Stat Naming Conventions

A list of possible stats that you gain from items, runes, or masteries can also be found in Data Dragon. You can find a list of stats gained by the item, rune, or mastery by searching for the stats field. Below are some tips when it comes to understanding what a stat means and how they are calculated:

- Mod stands for modifier.
- An **"r"** at the beginning of the stat means those stats can be found on runes.
- Displaying flat vs. percentage vs. per 5 etc. is case-by-case. it will always be the same for a given stat. For example, PercentAttackSpeedMod will always be multiplied by 100 and displayed it as a percentage.
- Stats are called flat if you add them together, and percent if you multiply them together.
- Tenacity from an item does NOT stack but tenacity from a rune DOES stack.

Item Assets

https://ddragon.leagueoflegends.com/cdn/9.19.1/img/item/**1001**.png
The number appended to the item filename corresponds to the item id. You can find a list of the items ids in the item data file.

Other

Summoner Spells

https://ddragon.leagueoflegends.com/cdn/9.19.1/data/en_US/**summoner.json**
https://ddragon.leagueoflegends.com/cdn/9.19.1/img/spell/**SummonerFlash**.png

Profile Icons

https://ddragon.leagueoflegends.com/cdn/9.19.1/data/en_US/**profileicon.json**
https://ddragon.leagueoflegends.com/cdn/9.19.1/img/profileicon/**685**.png

Minimaps

https://ddragon.leagueoflegends.com/cdn/6.8.1/img/map/**map11**.png
The number appended to the map filename corresponds to the map id. You can find a list of the map ids in the Map Names section of Game Constants.

Sprites

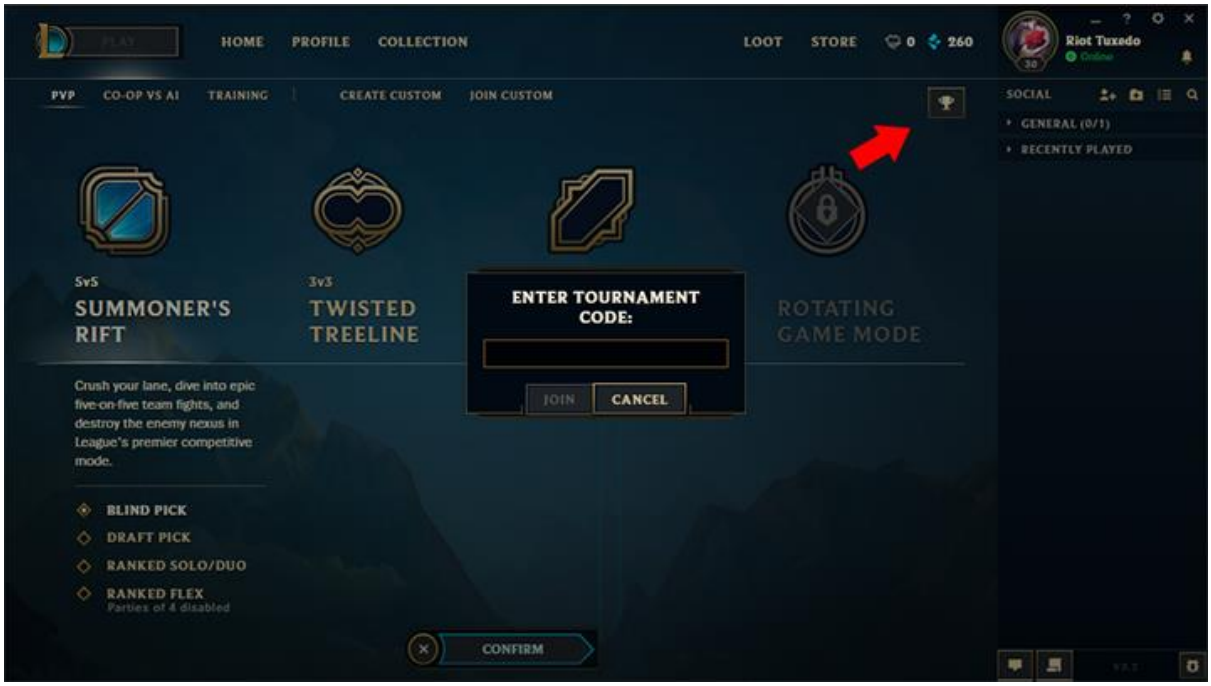
https://ddragon.leagueoflegends.com/cdn/9.19.1/img/sprite/**spell0**.png

Scoreboard Icons (version 5.5.1)

https://ddragon.leagueoflegends.com/cdn/5.5.1/img/ui/**champion.png**
https://ddragon.leagueoflegends.com/cdn/5.5.1/img/ui/**items.png**
https://ddragon.leagueoflegends.com/cdn/5.5.1/img/ui/**minion.png**
https://ddragon.leagueoflegends.com/cdn/5.5.1/img/ui/**score.png**
https://ddragon.leagueoflegends.com/cdn/5.5.1/img/ui/**spells.png**

Tournament API

League of Legends leverages custom game lobbies to support developers that want to build Tournaments for players. Custom games can use Tournament Code that help you quickly and easily get players into private custom game lobbies with preset game settings, such as map and pick type. At the completion of each Tournament Code enabled game, the results will be forwarded automatically (HTTP POST) to a URL specified by the tournament developer.



The Tournaments API allows you to:

1. Register tournament providers and tournaments in a specific region/shard and its callback URL.
2. Generate tournament codes for a given tournament.
3. Receive game results in the form of an automatic callback (HTTP POST) from League of Legends servers whenever a game created using tournament code has been completed.
4. Use match identifier (matchID) received in the callback to pull full stats data for the given match.
5. Pull end of game data based on given tournament code in case the callback is never received.
6. Query pre-game lobby player activity events for a given tournament code.

Best Practices

To preserve the quality of the tournaments service, your Tournaments API Key may be revoked if you do not adhere to the following best practices:

- Respect the rate limit for your Tournament API Key and implement logic that considers the headers returned from a 429 Rate Limit Exceeded response.

- Implement logic to detect unsuccessful API calls and back off accordingly. Please notify us if you believe your application is working correctly and you are receiving errors, but do not continue to slam the tournaments service with repeatedly unsuccessful calls.
- Generate tournaments and tournament codes only as needed in production and development. Please don't create 1,000 tournament codes for a 10 game tournament. As a reminder, you can always create additional tournament codes as your tournament grows.
- Tournaments and tournament codes should be generated within a reasonable time in relation to the event. Do not pre-create tournaments and tournament codes at the start of the year and use them as the year progresses, but rather generate the tournament and codes as the event is announced and participants sign up.

Tournament API Notes

When working with the Tournament API, keep the following in mind:

- Tournament providers are strongly associated with API keys, regenerating an API key will require a new provider.
- Though tournament codes can be re-used to generate additional lobbies. For best results with callbacks and Match-v4 lookups, create a single match with a tournament code.
- Lobby events should only be used to audit Tournament matches as needed. In rare cases lobby events may get dropped. Using lobby events to programmatically progress a tournament or to forfeit participants is not advised.
- Tournaments will expire if there are no active codes associated with the tournament. Tournament codes are eligible for expiration three months after they are generated. As tournaments and their codes can expire, creating them as close to the event as possible ensures no disruptions. For the best results:
    - Create a tournament no more than a week before the start of the first match.
    - Upon creation of the tournament, generate a code to ensure the tournament has an active tournament code associated with it (thereby making ineligible for cleanup).
    - Tournament codes should be generated as needed, not all at once at the start of the event.

Use Case Example

Presume there is a tournament website created for League of Legends players that does the following:

- Announces tournament and rules
- Registers players/teams
- Generates/renders tournament brackets
- Seeds registered teams across the brackets
- Sends invites for matched teams to play their games
- Collects end of game results from team captains

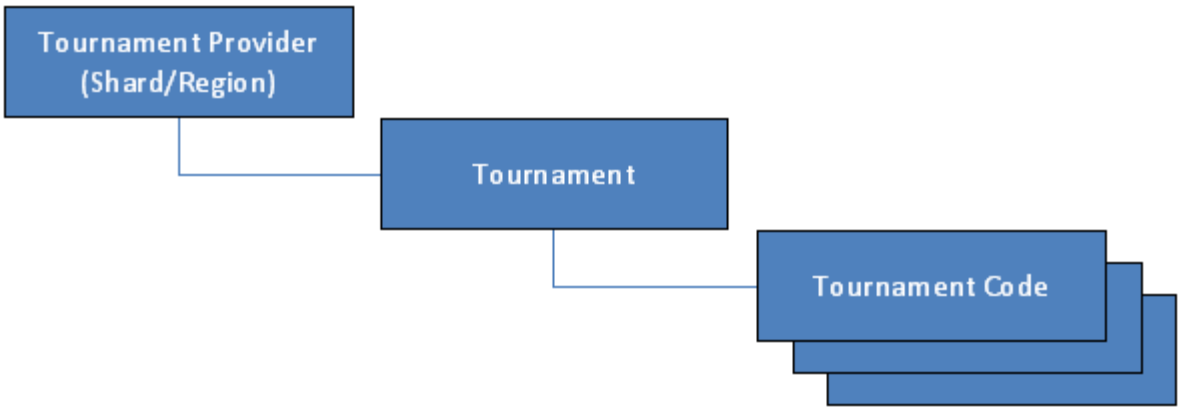
- Provides new matches for teams that advance
- Officiates for situations when something goes wrong (no show, etc)

The Tournaments API is designed to help automate the last four mentioned functions of tournament websites.

It is recommended to register a tournament provider (specifying region/shard and URL for results) well in advance and do a full loop testing to ensure everything is setup properly for your web service.

Tournaments API Structure

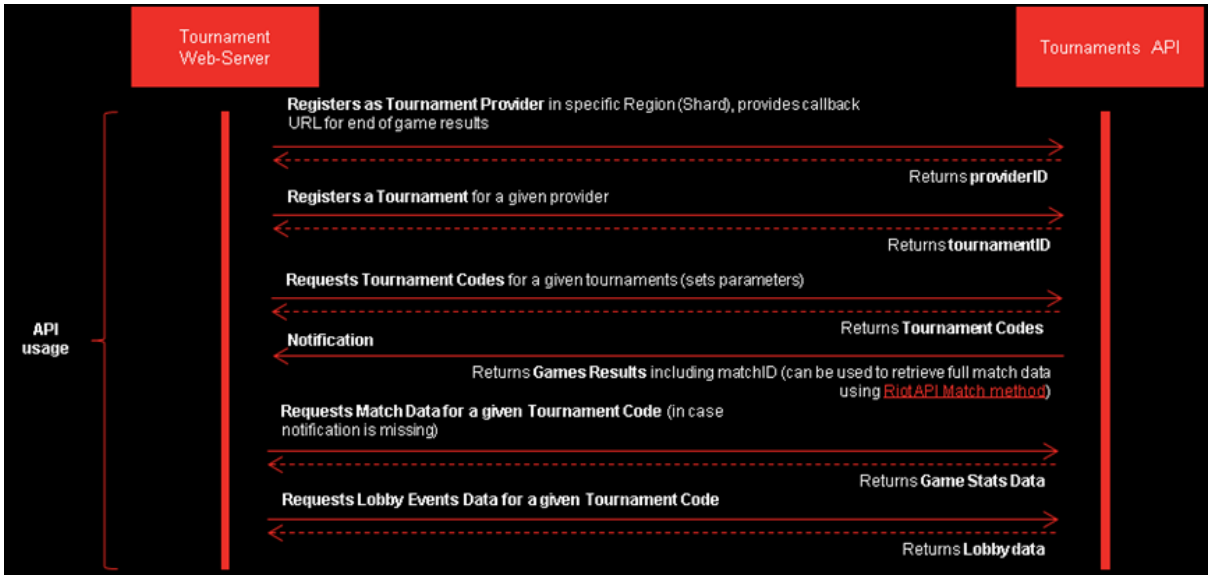
The Tournaments API introduces a simple parent-child data structure to ensure data model consistency:



Tournaments API Methods

Access to the Tournaments API provides several new methods that can be viewed on the API Reference page. You should explore every method to get more information on actual usage including the format and description of parameters you can supply.

See the diagram below for a full overview of all methods and their functionality:



Generating Tournament Codes

To generate tournament codes:

1. Use /lol/tournament/v5/providers API endpoint to register as a provider in specific region while also setting a URL to be used for receiving game results notifications (HTTP POST). Returns providerID.
2. Use providerID to register a tournament for given Tournament Provider.
   Receive tournamentID in return.
3. Use tournamentID to generate one or more tournament codes for a given tournament using specific game settings, such as map, spectator rules, or pick type.
    - A tournament code should only be used to create a single match. If you reuse a tournament code, the server callback will not return stats for each match.
    - The method to generate tournament codes will return up to 1,000 tournament codes at a time. If needed, additional calls to this method can be made to create additional tournament codes.
    - Stale or unused tournament codes may be purged after a period of inactivity.

Tournament organizers can generate all the tournament codes they need in advance or generate them as necessary per each phase as shown below:

Server Callback

When a game created using tournament code has completed, the League of Legends servers will automatically make a callback to the tournament provider's registered URL through HTTP POST. Below are a couple notes about how the server callback works.

The provider registration and callback mechanism are relatively inflexible. For best results, use one of the valid generic top level domains (gTLDs) listed below and use HTTP over HTTPS for your callback URL while using the metaData field to validate callbacks.

Port Restrictions
The server callback supports http (port 80) and https (port 443) however Certificate Authorities (CA) approved after Jan 29, 2012 aren't supported. The callback server won't perform a callback if it is unable to validate an SSL cert issued by an unknown CA (and therefore doesn't trust).

Domain Restrictions
Only valid gTLDs approved by ICANN before March 2011 are considered valid. This excludes newer gTLDs such as (.mail, .xxx, .xyz, etc.)

Valid gTLDs (approved before March 2011)
aero, asia, biz, cat, com, coop, info, jobs, mobi, museum, name, net, org, pro, tel, travel, gov, edu, mil, int

Valid Country Code TLDs
ac, ad, ae, af, ag, ai, al, am, an, ao, aq, ar, as, at, au, aw, ax, az, ba, bb, bd, be, bf, bg, bh, bi, bj, bm, bn, bo, br, bs, bt, bv, bw, by, bz, ca, cc, cd, cf, cg, ch, ci, ck, cl, cm, cn, co, cr, cu, cv, cx, cy, cz, de, dj, dk, dm, do, dz, ec, ee, eg, er, es, et, eu, fi, fj, fk, fm, fo, fr, ga, gb, gd, ge, gf, gg, gh, gi, gl, gm, gn, gp, gq, gr, gs, gt, gu, gw, gy, hk, hm, hn, hr, ht, hu, id, ie, il, im, in, io, iq, ir, is, it, je, jm, jo, jp, ke, kg, kh, ki, km, kn, kp, kr, kw, ky, kz, la, lb, lc, li, lk, lr, ls, lt, lu, lv, ly, ma, mc, md, me, mg, mh, mk, ml, mm, mn, mo, mp, mq, mr, ms, mt, mu, mv, mw, mx, my, mz, na, nc, ne, nf, ng, ni, nl, no, np, nr, nu, nz, om, pa, pe, pf, pg, ph, pk, pl, pm, pn, pr, ps, pt, pw, py, qa, re, ro, rs, ru, rw, sa, sb, sc, sd, se, sg, sh, si, sj, sk, sl, sm, sn, so,

sr, st, su, sv, sy, sz, tc, td, tf, tg, th, tj, tk, tl, tm, tn, to, tp, tr, tt, tv, tw, tz, ua, ug, uk, um, us, uy, uz, va, vc, ve, vg, vi, vn, vu, wf, ws, ye, yt, yu, za, zm, zw

- The callback from the League of Legends server relies on a successful response from the provider's registered URL. If a 200 response is not detected, there is a retry mechanism that will make additional attempts. In the rare occasion that a callback is not received within 5 minutes, you can assume the callback failed.
- If you need to change your provider callback URL, register a new provider but remember tournaments generated with the old provider will continue to make callbacks to the old provider callback URL.

When a game created using Tournament Code has completed, the League of Legends servers will automatically make a callback to the tournament provider's registered URL via HTTP POST. Below are a couple notes about how the server callback works.

If you are having trouble debugging your logic, use the following cURL to mimic the behavior of the callback:

curl <callback_url> -X POST -H "Content-Type: application/json" -d '<response_body>'

Below is a sample JSON response returned by the League of Legends servers when a callback is made:


```
{
  "startTime": 1234567890000,
  "shortCode": "NA1234a-1a23b456-a1b2-1abc-ab12-1234567890ab",
  "metaData": "{\"title\":\"Game 42 - Finals\"}",
  "gameId": 1234567890,
  "gameName": "a123bc45-ab1c-1a23-ab12-12345a67b89c",
  "gameType": "Practice",
  "gameMap": 11,
  "gameMode": "CLASSIC",
  "region": "NA1"
}
```


Lobby Events

In addition to game stats related methods, the lobby-events/by-code/{tournamentCode} method that can help query pre-game lobby events. This is useful for building tournament administration system and be able to detect whether a game for a given tournament code started normally. This call can be made both after the match for the full timeline and anytime during the lobby phase for a timeline of events up to that moment. Below is an example of the JSON returned for lobby events:


```
{
  "eventList": [
```

```
{
 "timestamp": "1234567890000",
 "eventType": "PracticeGameCreatedEvent", //Lobby Created
 "summonerId": "12345678"
},
{
 "timestamp": "1234567890000",
 "eventType": "PlayerJoinedGameEvent", //Player Joins Lobby
 "summonerId": "12345678"
},
{
 "timestamp": "1234567890000",
 "eventType": "PlayerSwitchedTeamEvent", //Player Switches Teams
 "summonerId": "12345678"
},
{
 "timestamp": "1234567890000",
 "eventType": "PlayerQuitGameEvent", //Player Leaves Lobby
 "summonerId": "12345678"
},
{
 "timestamp": "1234567890000",
 "eventType": "ChampSelectStartedEvent"  //Champ Select Begins
},
{
 "timestamp": "1234567890000",
 "eventType": "GameAllocationStartedEvent"  //Loading Screen Begins
},
{
 "timestamp": "1234567890000",
 "eventType": "GameAllocatedToLsmEvent"  //Game Begins
```

```
    }
  ]
}
```

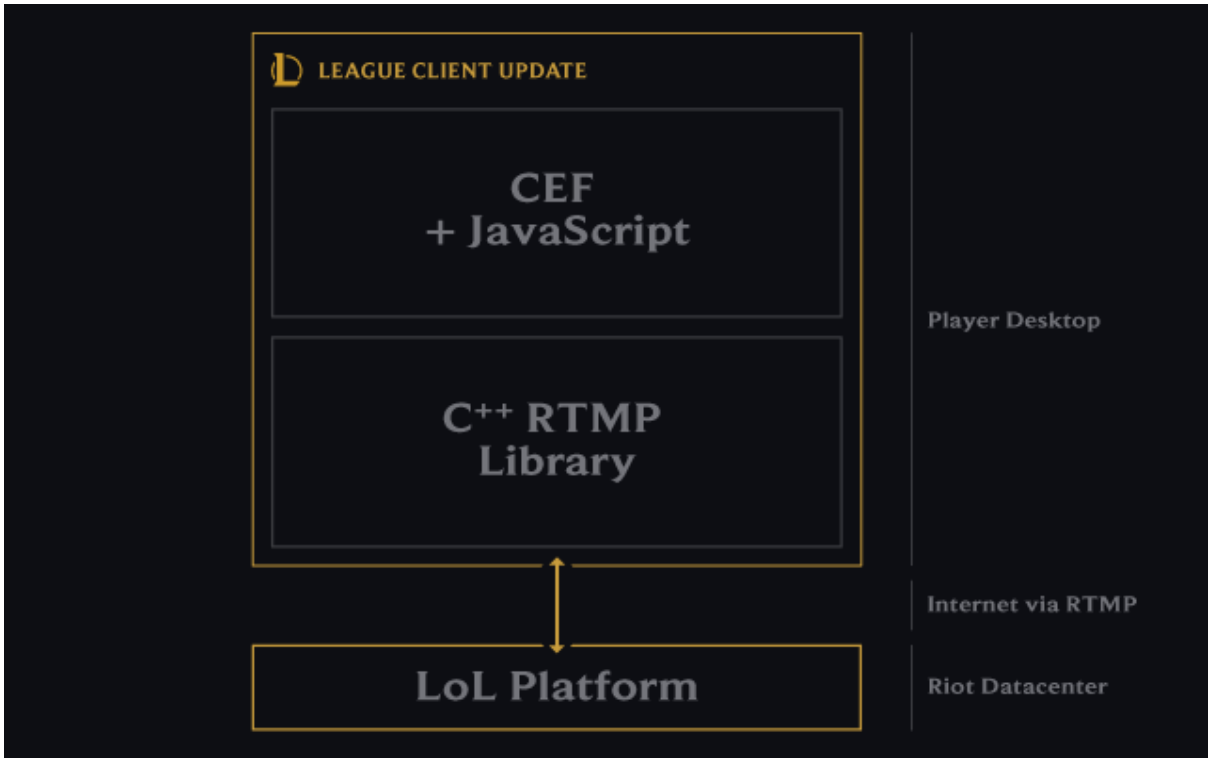
League Client API

What is the League Client API?

In an article on the Riot Games Engineering Blog, there's an image that is useful for defining what we're classifying as "League Client APIs".

Specifically, we're referring to a set of protocols that the Chromium Embedded Framework (CEF) uses to communicate with a C++ Library that in turn communicates with the League of Legends platform. As you'll notice, the communications between the C++ library and the CEF all occur locally on your desktop. This is the League Client API. This service is not officially supported for use with third party applications.

NOTE: We provide no guarantees of full documentation, service uptime, or change communication for unsupported services. This team does not own any components of the underlying services, and will not offer additional support related to them.



What's next

Whether you're combining the Riot Games API and League Client API, or doing something by only using the League Client endpoints, we need to know about it. Either create a new application or leave a note on your existing application in the Developer Portal. We need to know which endpoints you're using and how you're using them in order to expand on current or future feature sets. If you have any questions please join the Developer Discord for help.

Game Client API

The Game Client APIs are served over HTTPS by League of Legends game client and are only available locally for native applications.

Root Certificate/SSL Errors

The League of Legends client and the game client use a self-signed certificate for HTTPS requests. To use the Game Client API, you can ignore these errors or use the root certificate to validate the game client's SSL certificate. If you are testing locally, you can use the following insecure CURL that will ignore the SSL certificate errors.

curl --insecure https://127.0.0.1:2999/swagger/v3/openapi.json

Swagger

You can request the Swagger v2 and OpenAPI v3 specs for the Game Client API with the following URLs:
https://127.0.0.1:2999/swagger/v2/swagger.json
https://127.0.0.1:2999/swagger/v3/openapi.json

Live Client Data API

The Live Client Data API provides a method for gathering data during an active game. It includes general information about the game as well player data.

Get All Game Data

The Live Client Data API has a number of endpoints that return a subset of the data returned by the /allgamedata endpoint. This endpoint is great for testing the Live Client Data API, but unless you actually need all the data from this endpoint, use one of the endpoints listed below that return a subset of the response.

GET https://127.0.0.1:2999/liveclientdata/allgamedata
Get all available data.

You can find a sample response here.

Endpoints

Active Player
GET https://127.0.0.1:2999/liveclientdata/activeplayer
Get all data about the active player.


```
{
   "abilities": {...},
   "championStats": {
     "abilityHaste": 0.00000000000000,
     "abilityPower": 0.00000000000000,
     "armor": 0.00000000000000,
     "armorPenetrationFlat": 0.0,
     "armorPenetrationPercent": 0.0,
```

```
  "attackDamage": 0.00000000000000,
  "attackRange": 0.0,
  "attackSpeed": 0.00000000000000,
  "bonusArmorPenetrationPercent": 0.0,
  "bonusMagicPenetrationPercent": 0.0,
  "cooldownReduction": 0.00,
  "critChance": 0.0,
  "critDamage": 0.0,
  "currentHealth": 0.0,
  "healthRegenRate": 0.00000000000000,
  "lifeSteal": 0.0,
  "magicLethality": 0.0,
  "magicPenetrationFlat": 0.0,
  "magicPenetrationPercent": 0.0,
  "magicResist": 0.00000000000000,
  "maxHealth": 0.00000000000000,
  "moveSpeed": 0.00000000000000,
  "physicalLethality": 0.0,
  "resourceMax": 0.00000000000000,
  "resourceRegenRate": 0.00000000000000,
  "resourceType": "MANA",
  "resourceValue": 0.00000000000000,
  "spellVamp": 0.0,
  "tenacity": 0.0
}
"currentGold": 0.0,
"fullRunes": {...},
"level": 1,
"summonerName": "Riot Tuxedo",
"riotId": "Riot Tuxedo#TXC1",
"riotIdGameName": "Riot Tuxedo",
```

"riotIdTagLine": "TXC1"
}

GET https://127.0.0.1:2999/liveclientdata/activeplayername
Returns the player name.

"Riot Tuxedo#TXC1"

GET https://127.0.0.1:2999/liveclientdata/activeplayerabilities
Get Abilities for the active player.

{
"E": {
"abilityLevel": 0,
"displayName": "Molten Shield",
"id": "AnnieE",
"rawDescription": "GeneratedTip_Spell_AnnieE_Description",
"rawDisplayName": "GeneratedTip_Spell_AnnieE_DisplayName"
},
"Passive": {
"displayName": "Pyromania",
"id": "AnniePassive",
"rawDescription": "GeneratedTip_Passive_AnniePassive_Description",
"rawDisplayName": "GeneratedTip_Passive_AnniePassive_DisplayName"
},
"Q": {
"abilityLevel": 0,
"displayName": "Disintegrate",
"id": "AnnieQ",
"rawDescription": "GeneratedTip_Spell_AnnieQ_Description",
"rawDisplayName": "GeneratedTip_Spell_AnnieQ_DisplayName"
},
"R": {
"abilityLevel": 0,
"displayName": "Summon: Tibbers",
"id": "AnnieR",

```
      "rawDescription": "GeneratedTip_Spell_AnnieR_Description",
      "rawDisplayName": "GeneratedTip_Spell_AnnieR_DisplayName"
   },
   "W": {
      "abilityLevel": 0,
      "displayName": "Incinerate",
      "id": "AnnieW",
      "rawDescription": "GeneratedTip_Spell_AnnieW_Description",
      "rawDisplayName": "GeneratedTip_Spell_AnnieW_DisplayName"
   }
}
```


GET https://127.0.0.1:2999/liveclientdata/activeplayerrunes
Retrieve the full list of runes for the active player.


```
{
   "keystone": {
      "displayName": "Electrocute",
      "id": 8112,
      "rawDescription": "perk_tooltip_Electrocute",
      "rawDisplayName": "perk_displayname_Electrocute"
   },
   "primaryRuneTree": {
      "displayName": "Domination",
      "id": 8100,
      "rawDescription": "perkstyle_tooltip_7200",
      "rawDisplayName": "perkstyle_displayname_7200"
   },
   "secondaryRuneTree": {
      "displayName": "Sorcery",
      "id": 8200,
      "rawDescription": "perkstyle_tooltip_7202",
      "rawDisplayName": "perkstyle_displayname_7202"
```

```
  },
  "generalRunes": [
    {
      "displayName": "Electrocute",
      "id": 8112,
      "rawDescription": "perk_tooltip_Electrocute",
      "rawDisplayName": "perk_displayname_Electrocute"
    },
    ...
  ],
  "statRunes": [
    {
      "id": 5007,
      "rawDescription": "perk_tooltip_StatModCooldownReductionScaling"
    },
    {
      "id": 5008,
      "rawDescription": "perk_tooltip_StatModAdaptive"
    },
    {
      "id": 5003,
      "rawDescription": "perk_tooltip_StatModMagicResist"
    }
  ]
}
```


All Players

GET https://127.0.0.1:2999/liveclientdata/playerlist

Retrieve the list of heroes in the game and their stats.


```
[
  {
    "championName": "Annie",
    "isBot": false,
```

```
        "isDead": false,
        "items": [...],
        "level": 1,
        "position": "MIDDLE",
        "rawChampionName": "game_character_displayname_Annie",
        "respawnTimer": 0.0,
        "runes": {...},
        "scores": {...},
        "skinID": 0,
        "summonerName": "Riot Tuxedo",
        "riotId": "Riot Tuxedo#TXC1",
        "riotIdGameName": "Riot Tuxedo",
        "riotIdTagLine": "TXC1",
        "summonerSpells": {...},
        "team": "ORDER"
    },
    ...
]
```


GET https://127.0.0.1:2999/liveclientdata/playerscores?riotId=
Retrieve the list of the current scores for the player.


```
{
    "assists": 0,
    "creepScore": 0,
    "deaths": 0,
    "kills": 0,
    "wardScore": 0.0
}
```


GET https://127.0.0.1:2999/liveclientdata/playersummonerspells?riotId=
Retrieve the list of the summoner spells for the player.


```
{
    "summonerSpellOne": {
        "displayName": "Flash",
```

```
      "rawDescription": "GeneratedTip_SummonerSpell_SummonerFlash_Description",
      "rawDisplayName": "GeneratedTip_SummonerSpell_SummonerFlash_DisplayName"
   },
   "summonerSpellTwo": {
      "displayName": "Ignite",
      "rawDescription": "GeneratedTip_SummonerSpell_SummonerDot_Description",
      "rawDisplayName": "GeneratedTip_SummonerSpell_SummonerDot_DisplayName"
   }
}
```


GET https://127.0.0.1:2999/liveclientdata/playermainrunes?riotId=
Retrieve the basic runes of any player.


```
{
   "keystone": {
      "displayName": "Electrocute",
      "id": 8112,
      "rawDescription": "perk_tooltip_Electrocute",
      "rawDisplayName": "perk_displayname_Electrocute"
   },
   "primaryRuneTree": {
      "displayName": "Domination",
      "id": 8100,
      "rawDescription": "perkstyle_tooltip_7200",
      "rawDisplayName": "perkstyle_displayname_7200"
   },
   "secondaryRuneTree": {
      "displayName": "Sorcery",
      "id": 8200,
      "rawDescription": "perkstyle_tooltip_7202",
      "rawDisplayName": "perkstyle_displayname_7202"
   }
}
```

GET https://127.0.0.1:2999/liveclientdata/playeritems?riotId=
Retrieve the list of items for the player.


```
[
  {
    "canUse": true,
    "consumable": false,
    "count": 1,
    "displayName": "Warding Totem (Trinket)",
    "itemID": 3340,
    "price": 0,
    "rawDescription": "game_item_description_3340",
    "rawDisplayName": "game_item_displayname_3340",
    "slot": 6
  },
  ...
]
```


Events
GET https://127.0.0.1:2999/liveclientdata/eventdata
Get a list of events that have occurred in the game.


```
{
  "Events": [
    {
      "EventID": 0,
      "EventName": "GameStart",
      "EventTime": 0.0325561985373497
    },
    ...
  ]
}
```


You can find a list of sample events here.

Game
GET https://127.0.0.1:2999/liveclientdata/gamestats
Basic data about the game.

{
"gameMode": "CLASSIC",
"gameTime": 0.000000000,
"mapName": "Map11",
"mapNumber": 11,
"mapTerrain": "Default"
}

Any of these endpoints that returned a summonerName, now return a RiotID shim over summonerName, and new fields called riotId, riotIdGameName and riotIdTagLine in structured responses. Any endpoints that took a SummonerName as a parameter now accepts only the riotId parameter. It attempts to match the name to RiotID first, then RiotIDGameName, then SummonerName (to maintain backwards compatibility until we can fully deprecate SummonerName).

Replay API

The Replay API allows developers to adjust the in-game camera during replays. League Director is an open source example of how a tool can leverage the Replay API.

Getting Started

By default the Replay API is disabled. To start using the Replay API, enable the Replay API in the game client config by locating where your game is installed and adding the following lines to the *game.cfg* file:

Example file location:
C:\Riot Games\League of Legends\Config\game.cfg

[General]

EnableReplayApi=1

Once you have enabled the Replay API, the game client will generate the Swagger v2 and OpenAPI v3 specs for the Replay API, which indicates the Replay API is usable.

Endpoints

GET https://127.0.0.1:2999/replay/game
Information about the game client process.

GET https://127.0.0.1:2999/replay/playback
Returns the current replay playback state such as pause and current time.

POST https://127.0.0.1:2999/replay/playback
Allows modifying the playback state such as play/pause and the game time to seek to. All values are optional.

GET https://127.0.0.1:2999/replay/render
Returns the current render properties.

POST https://127.0.0.1:2999/replay/render
Allows modifying the current render properties. All values are optional.

GET https://127.0.0.1:2999/replay/recording
Returns the current status of video recording. Poll this resource for progress on the output.

POST https://127.0.0.1:2999/replay/recording
Post to begin a recording specifying the codec and output filepath. Subsequent GET requests to this resource will update the status.

GET https://127.0.0.1:2999/replay/sequence
Returns the sequence currently being applied.

POST https://127.0.0.1:2999/replay/sequence
Post to apply a sequence of keyframes that the replay should play. Post an empty object to remove the sequence.

Working with LoL APIs

Game Constants

When looking up specific seasons, queues, maps, and modes it is important to use the correct IDs.

Seasons

Season IDs are used in match history to indicate which season a match was played. A full list of season ids can be found in *seasons.json*.


```
[
 {
  "id": 0,
  "season": "PRESEASON 3"
 },
 ...
]
```


Queue IDs

Queue IDs show up in several places throughout the API and are used to indicate which kind of match was played. A full list of queue ids can be found in *queues.json*.

Note: In early 2022, URF (previously queueId 900) was divided into separate queues— ARURF (queueId 900) and Pick URF (queueId 1900). All Pick URF games from before this distinction will still be in queueId 900.


```
[
 {
  "queueId": 0,
  "map": "Custom games",
```

```
  "description": null,
  "notes": null
 },
 ...
]
```


Maps

Map IDs are used in match history to indicate which map a match was played. A full list of map IDs can be found in *maps.json*.


```
[
 {
  "mapId": 1,
  "mapName": "Summoner's Rift",
  "notes": "Original Summer variant"
 },
 ...
]
```


Game Modes

A full list of game modes can be found in *gameModes.json*.


```
[
 {
  "gameMode": "CLASSIC",
  "description": "Classic Summoner's Rift and Twisted Treeline games"
 },
 ...
]
```


Game Types

A full list of game types can be found in *gameTypes.json*.


```
[
 {
  "gameType": "CUSTOM_GAME",
  "description": "Custom games"
```

},

...

]

Ranked Info

Queue Types

The League endpoints return a field called queueType that indicates what map/mode a player played. Depending on the queueType, the highestTierAchieved field returns the highest ending tier for the previous season from a group of ranked queues.

Here is a list of all of the queueType and highestTierAchieved for each.

Summoner's Rift

Unranked

RANKED_SOLO_5x5

RANKED_TEAM_5x5

Ranked Solo/Duo

RANKED_SOLO_5x5

Ranked Team 5x5

RANKED_TEAM_5x5

Other Maps

If a match is not played on Summoner's Rift, the highestTierAchieved field will return the highest ending tier for the previous season from any ranked queue.